DEEP REINFORCEMENT LEARNING TO SOLVE THE ORIENTEERING PROBLEM

Krystal A Coleman
Department of Industrial & Systems Engineering
SUNY, University at Buffalo
Buffalo, NY 14260, USA
kc64739@buffalo.edu

1 INTRODUCTION

Orienteering was established as a military training exercise in Sweden in the late 19th century before it became a formal outdoor competitive sport in the early 20th century. Since then, the sport has grown in popularity (Orienteering Queensland, n.d.). In 2024, the International Orienteering Foundation will host its annual world championships among its 78 member nations. In the sport of orienteering, participants must visit several locations in the shortest possible time, using only a map and compass as guides. In score orienteering, commonly referred to as score-O, each location is preassigned a score which participants score by visiting. The participants have limited time and must return to a common location after navigating to a subset and sequence of locations to maximize their score. Rogaining, another orienteering sport, shares the basis of score-O; a map of appraised locations exists, however, rogaining events tend to be longer and competitors are given the map some time ahead of the competition to plan their route and strategy. (Tsiligirides, 1984) used the name Orienteering Problem (OP) to identify the mathematical formulation of the set of problems which require planning a route of stops in advance to maximize reward within a time limit.

Many real-world problems have been identified as orienteering problems and can be modeled using this formulation (see section 2). OPs differs from Traveling Salesman Problems (TSPs) where all locations must be visited. The goal of maximizing reward with limited capacity is akin to the Knapsack Problem (KP). The OP is NP-Hard (Golden et al., 1987). We consider applications of the OP that require good solutions to be produced quickly and where agents start and end their tours at the same location (referred to by Mansini et al. (2006) as the tour orienteering problem). Examples include scheduling facility maintenance based on submitted requests. It is important to provide regenerate routes for technicians as circumstances change on short notice, for example, if a new, high-priority service request is entered into the system. We hope to extend the methodology used in this project to the Team Orienteering Problem (TOP) and TOP with time windows (TOPTW) in the future. These are at least just as hard to solve as the OP. Chao et al. (1996b) defined the TOP as a multi-agent version of the OP where agents collaborate to maximize the total reward. In this, each agent must be assigned a sequence of locations to visit within the time limit to maximize their combined score. A natural extension of this is the TOPTW where agents can only visit each location within a specific time window.

A highly adaptable and versatile solution method is necessary to solve the OP. We discuss the work of researchers to accomplish this task in section3. We discuss in section 4 how we use a deep reinforcement learning model (DRL) to solve the OP. We compare our solution to those for benchmark OP provided by Chao et al. (1996a) and Tsiligirides (1984) and compiled in KU Leven's Orienteering Problem Library (OPLIB) in section 6.

2 BACKGROUND

Before proposing a model to solve this problem we first describe a linear program, inspired by Vansteenwegen et al. (2011), which is used to find an exact solution to the problem. This formulation is important to understand the constraints considered in our RL model.

Consider a graph $G(V, A)$ with the set of vertices V labeled such that $V = \{0, 1, 2, ..., n, n+1\}$. Vertex 0 represents the start location of each agent's path and $n+1$ represents the end location of each agent's path so that there are n possible vertices that agents can stop at. We will call these service vertices. We assume that once an agent stops at a vertex its full reward is collected. We hypothesize that in large instances a reinforcement learning model would be able to produce a sufficiently good solution within a more reasonable time than the linear program.

2.1 Mixed Integer Linear Program

Parameters

V	Set of all vertices
N	Set of vertices to be visited,
	$N \subseteq V$ such that $N = \{x \in V \mid x \neq 0, x \neq n+1\}$ and $\lvert N \rvert = n$
T_{max}	Time limit on the route
r_i	Score assigned to each vertex $i \in V$
t_{ij}	Time it takes to travel from vertex $i \in V$ to vertex $j \in V$

Decision Variables

x_{ij}	$\begin{cases} 1 & \text{if a stop at vertex } i \in V \text{ directly precedes a stop at vertex at } j \in V \\ 0 & \text{otherwise} \end{cases}$
u_i	the position of vertex $i \in V$ in the path

Formulation

$$\max \quad \sum_{i=1}^{n} \sum_{j=1}^{n+1} r_i x_{ij} \tag{1}$$

$$\text{subject to} \quad \sum_{j=1}^{n+1} x_{0,j} = 1 \tag{2}$$

$$\sum_{i=0}^{n} x_{i,n+1} = 1 \tag{3}$$

$$\sum_{i=0}^{n} x_{i,k} \leq 1, \qquad \forall k \in N \tag{4}$$

$$\sum_{j=1}^{n+1} x_{k,j} \leq 1, \qquad \forall k \in N \tag{5}$$

$$\sum_{i=0}^{n} \sum_{j=1}^{n+1} t_{i,j} x_{i,j} \leq T_{max}, \tag{6}$$

$$u_i \geq 1, \qquad \forall i \in N \tag{7}$$

$$u_i \leq n, \qquad \forall i \in N \tag{8}$$

$$u_i - u_j + 1 \leq n(1 - x_{ij}) \qquad \forall i, j \in N \tag{9}$$

$$u_i \in \mathbb{Z}^+ \qquad \forall i \in V \tag{10}$$

$$x_{ij} \in 0, 1 \qquad \forall i \in V, j \in V \tag{11}$$

Our objective is to maximize the total reward collected 1. All tours start at vertex i 2 4 and end at vertex N 3 5 such that the position of other vertices is between 0 7 and n 8. The tour must be completed within a time limit 6. We prevent subtours with constraint 9. Our variables u_i are non-negative integers and x_{ij} are binary.

3 LITERATURE REVIEW

3.1 ORIENTEERING PROBLEM

Before it was called the orienteering problem, many researchers such as Golden et al. (1984) studied similar problems such as routing delivery vehicles to replenish inventory of gas or fuel when it is imperative that inventories be kept above a minimum level. The prize collecting TSP (Balas, 1989) is also similar to the OP though there is no explicit time constraint and the objective itself incorporates the goal of minimizing the total travel cost. The selective traveling salesman problem (STSP) (Laporte & Martello, 1990) considers that not all vertices can be visited due to an upper limit on traveling costs. The STSP mandates that some vertices are visited and in the OP the compulsory vertices are those the tours start and end at.

Both branch and bound Laporte & Martello (1990) Ramesh et al. (1992) and branch and cut (Fischetti et al., 1998) (Gendreau et al., 1998) algorithms can be used to exactly solve the orienteering problem. These methods work well on smaller instances but struggle as the scale of instances grow past a few hundred. Metaheuristic methods have been proposed which tend to scale better and perform well much faster but these methods sacrifice optimality in order to achieve this. Vansteenwegen et al. (2011) provide a survey of the OP, its most common variants and their solution methods.

3.2 TEAM ORIENTEERING PROBLEM WITH TIME WINDOWS

Boussier et al. (2007) use a Branch and Price algorithm to solve the TOP and the selective vehicle routing problem with time windows (SVRPTW), a generalization of the TOPTW in which the route has a limited time length and vehicles themselves have limited capacity for travel. Branch and price algorithms consume a lot of memory and are computationally expensive. Boussier et al.tested this method on instances with up to 100 service vertices. In that hybridized solution method which combines branch and bound and column generation, once the sub-problem is solved, there are two branches generated on a fractional variable that either forbid or mandate visiting that vertex. Results from their testing show that as the time limit increases on the TOP, the problems are harder to solve and also, that a combination of acceleration techniques effectively increases the speed of the solution. To accelerate this algorithm Boussier et al. modified a tree search heuristic (Harvey & Ginsberg, 1995) previously used to accelerate branch-and-price on the VRPTW (Feillet et al., 2007). Tae & Kim (2015) use Boussier's Branch and Price algorithm to optimally solve the TOPTW instances from Righini & Salani (2009). Their results also show an increase in solution time as the number of agents in the team increases.

The modification of conventional local search heuristics, previously applied to the TOP, has provided high-quality solution methods to the TOPTW. Vansteenwegen et al. (2009) develop an efficient iterated local search heuristic. They define an insertion step to add vertices one by one to an agent's path and a shake step to move from local optima. Their heuristic performs well in a short amount of time. Moreover, their contribution to publicly available benchmark datasets used in this project is invaluable.

(Montemanni et al., 2011) use the Ant Colony Metaheuristic with great results. Gedik et al. (2017) propose a constraint programming approach and evaluate its performance on the benchmark instances used by Tae & Kim (2015). Gedik et al.'s constraint programming approach outperforms other heuristics on some instances. Gunawan et al. (2015) solves the TOPTW with an improved iterated local search adapted from Vansteenwegen's. With existing solutions, there seems to be some trade-off between higher-quality solutions and the speed of generating these solutions. This work proposes to use reinforcement learning as a method to excel in both regards.

3.3 REINFORCEMENT LEARNING FOR COMBINATORIAL OPTIMIZATION

Gama & Fernandes (2021) apply reinforcement learning to the Orienteering Problem with Team Windows, incorporating pointer networks into their network architecture which provides the inspiration for this project. There has been other great success in the application of deep learning methods to combinatorial optimization problems ((Kool et al., 2018), (Nazari et al., 2018) (Bello et al., 2016)). See Bengio et al. (2021) for a comprehensive review.

Mnih et al. (2015) propose the Deep Q Network to estimate the value of Q-values. Q-learning Watkins & Dayan (1992) is widely used in tabular reinforcement learning environments where states and actions are discrete and countably finite. The Q-value is the total reward an agent can expect to receive by taking an action in a particular state and then continuing to follow a given policy π afterwards.

$$Q^{\pi}(s_t, a_t) = r(s_t, a_t) + \gamma E[V^{\pi}(s_{t+1})]$$

Van Hasselt et al. (2016) builds on that work and creates Double Deep Q Learning. We further discuss this algorithm in Section 4. Cappart et al. (2021) use a hybridized algorithm of reinforcement learning and constraint programming to solve the TSPTW. This project leverages deep reinforcement learning inspired by these two papers to solve the OP. We choose deep reinforcement learning because of its adaptability and scalability. We incorporate Graph Attention Networks (Veličković et al., 2017) to encode the graphs within our Double Deep Q Network.

4 METHODOLOGY

4.1 DATA

From the OPLIB we use 26 instances as provided by (Chao et al., 1996a); for training, 18 instances are used and for testing and validation both use 4 instances. In other words, approximately 70% of data is used for training, 15% for validation and 15% for testing. All the instances used in this project had 66 vertices.

Deep learning generally requires a lot of input data. In this project we specifically perform deep Q learning using a target network and replay buffer. The replay buffer retains a long term memory of experiences the agent has had and provides data that we sample from for learning. We fill the replay buffer to capacity b at the beginning of training by collecting complete trajectories from each OP instance. A trajectory is a sequence of transitions (s_t, a_t, r_t, s_{t+1}) as is convention in reinforcement learning. In each time-step t the agent observes the environment. In the context of this project our environment is deterministic and the state the agent is in at a discrete time s_t is simply the adjacency matrix A of the agent's orienteering tour so far. Given that the agent is in state s_t it must decide on an appropriate action a_t which is the next vertex to visit in the orienteering instance. The agent collects reward r_t from the environment based on the action selected. The orienteering problem has rewards inherent to the nature of the problem and so the reward collected is simply the score assigned to each vertex r_i (see section 2). The next state the agent is in s_{t+1} is the same as a_t, that is, the vertex the agent decided to travel to. For bootstrapping purposes in our learning algorithm we also keep record of whether the episode ends due to termination or truncation with indicators c and l respectively. For utility it is also important to preserve structural information on the instance I that the agent's transition was taken from such as the complete record of time between vertices and reward of each vertex. The complete transition stored to the buffer includes the class object I which stores these attributes. The replay buffer is then a list-like object, a deque, which stores named tuples of transitions. When a new transition is to be added to the beginning of this, if maximum capacity will be violated, an old transition is removed from the end.

Transition(
state : s_t,
action : a_t,
reward : r_t,
next_state : s_{t+1},
terminated : c_t,
truncated : l_t,
instance : I)

Figure 1: Transition Named Tuple

The decision as to which action to take in each time-step a_t follows the ϵ-greedy rule where with some probability $1 - \epsilon$ the agent chooses the action with the greatest Q-value. This is the short term, greedy, exploitation of the environment. With probability, ϵ the agent can instead choose to explore the environment, choosing uniformly between the available actions. The pseudocode for this procedure is shown in Alg. 1 It is critical in the orienteering problem that subtours are not formed (see section 2) and so we mask the vertices already visited so the agent can only choose from not yet visited vertices, regardless of whether it is currently exploiting or exploring the environment. The initial value of ϵ and probability of exploration is 1 as we want to encourage the agent to fully explore the environment at the beginning of training. As training continues we reduce the value of ϵ assuming that the accuracy the Q-value estimation by the policy network is increasing. The rate at which we reduce ϵ after each episode, γ is a hyparameter that is tuned in our experiments.

Algorithm 1: ϵ-greedy Pseudocode

Input: ϵ

function Choose Action
    p = random
    if $p < \epsilon$ **then**
        Get $visited$, a list of vertices in this instance already visited by the agent
        $options = actions.delete(visited)$
        $action = random.choice(options)$
    else
        Mask the output by replacing the Q values in the policy network's output with $-\infty$ in the indices of visited vertices
        $action = argmax(otutput_{masked})$
return *action*

4.2 Learning Algorithm

The goal of our RL model is to accurately estimate the value of being in state s_t and taking action a_t, $Q(s, a)$. We combine the concepts of DDQN and GAT (see Section 3) to create a model to perform this estimation. The DDQN model was chosen for its stability and ability to reduce the problem of overestimation of Q values which exists in the base DQN. DDQN achieve this by using the policy network to choose the best action and the target network to evaluate the (Q-value) of the action rather than using the target network to perform both functions (as in DQN). In empirical studies DDQN has outperformed DQN and achieved faster convergence (Van Hasselt et al. (2016).

Before we begin training we initialize target (offline) and policy (online) networks Q_t and Q_p with parameters ϕ and ψ. The policy network initially has uniform weights so that each action has equal probability of being chosen. We use Van Hasselt et al. (2016) Double Deep Q Learning Algorithm shown in Alg. 2 to learn in each episode of the reinforcement learning environment. The procedure shown is repeated until convergence. We use the Adam optimizer and l2 regularization, the value of which is tuned in experiments. We determine convergence using standard early stopping criteria, if there is no decline in loss beyond an established δ across p consecutive episodes. Every n episodes $\phi = \psi$. To facilitate learning we customize an RL environment for each OP instance using Farama's Gymnasium module.

4.3 Testing and Validation

For testing and validation we complete a trajectory with the agent choosing actions according to the output of the DQN (exploitation only) and track the total rewards received in each instance. Validation is done at the end of each episode so we track how the agent's total rewards in each OP instance change as training continues. For final testing, we report the agent's total rewards in each OP instance and report how that differs from the optimal objective value.

Algorithm 2: Double Deep Q-Learning Algorithm with Replay Buffer

Input: Replay buffer of size b

function Episode
  Reset environment for the instances and get initial state s_0
  while *episode not done* **do**
    for $instance \in instances$ **do**
      Choose action a_t based on $Q_p(s_t)$ and ϵ-greedy rule
      Observe r_t, s_{t+1}, c (1 if tour is complete, 0 otherwise) and l (1 if the agent has run out of time, 0 otherwise)
      Save transition $\{s_t, a_t, r_t, s_{t+1}, c_t, l_t\}$ to the replay buffer
      Remove old transition from the replay buffer
      $s_t = s_{t+1}$
    for $j = 1 \in k$ **do**
      Randomly sample a mini-batch of m transitions (s, a, r, s') from the buffer
      `/* Perform remaining steps in this for loop using batch processing */`
      Predict Q-value for state s: $Q_p(s, a)$, the q-value of being in state s and taking action a as done in the sampled transition
      Get $a^* = \arg\max Q_p(s', a)$, the best action that could be taken in state s' according to policy network
      Estimate Q-value of state s' and taking action a^* according to target network: $Q_t(s', a^*)$
      Find target Q-value for current state s: $r + \gamma \cdot (1 - c) \cdot Q_t(s', a^*)$
      Calculate mean square error (MSE) between prediction and target (batch process)
      Backpropagate loss in the online model
    if *c=1 or l=1 for all instances* **then**
      Episode is done $\epsilon = \gamma * \epsilon$

5 Model Architecture

The first inputs to our DDQN (Fig. 3) are the node embeddings of the state of each sample in the mini-batch. In the DQN these node representations are pooled into a single representation for each sample in the minibatch. These graph embedding are then passed to two fully connected layers, each followed by a non-linear activation function. We use a softmax activation function on the output to produce the probability with which each action should be selected.

5.1 Node Embeddings

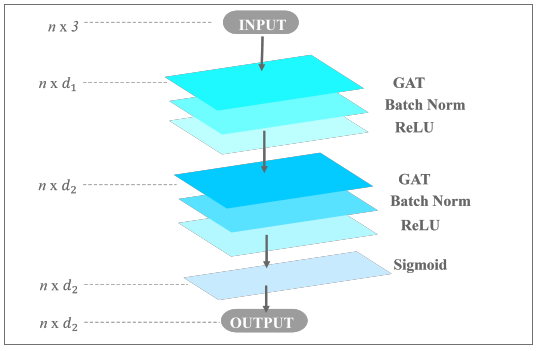


Figure 2: Neural Network for Node Embedding

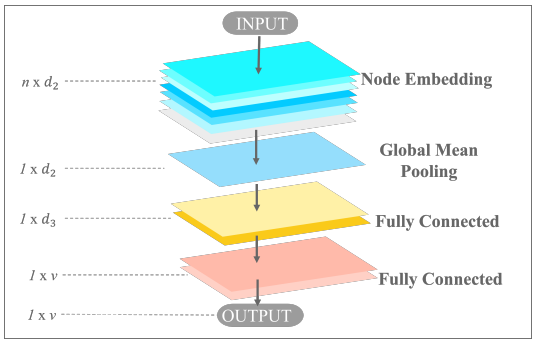


Figure 3: Neural Network for Node Embedding

GRAPH ATTENTION LAYERS

We use the GAT module in Pytorch-Geometric as constructed in Veličković et al. (2017). In a traditional GNN layer, to calculate the hidden representation for a node h_i we use a non-linear function of the product of a matrix of learnable weights W and the feature vector X_j of nodes j in the neighborhood of i. In the context of OP tours, a node's neighbors will be the nodes that directly precede and succeed it in the tour.

$$h_i = \sum_{j \in N(i)} \mathbf{W} * \vec{X_j} \tag{12}$$

GAT builds on this with attention mechanisms which learn how important a node j is to node i. The attention weight for a node j connected to i, a_{ij} can be written as a function of the transformation of i's and j's feature vectors. This is normalized over all $j \in N(i)$ using a softmax activation function.

$$a_{ij} = \frac{e^{f\left(\mathbf{W}\vec{X_i}, \mathbf{W}\vec{X_j}\right)}}{\sum_{j \in N(i)} e^{f\left(\mathbf{W}\vec{X_i}, \mathbf{W}\vec{X_j}\right)}} \tag{13}$$

We combine the messages passed from all $j \in N(i)$ to i to update the representation of i and produce the output of the GAT layer h'_i.

$$h'_i = \sum_j a_{ij} * h_i \tag{14}$$

The first GAT layer as shown in Fig. 2 has input for each node X_i with dimension 3 and output with dimension $d_1 = 128$. This dimension is not changed by normalization and applying an activation function. The second GAT reduces dimension from 128 to 64. Each GAT layer performs dropout on input features with probability dr.

BATCH NORMALIZATION

We use batch normalization **?** in our node embedding model as used in the original paper on GATs. As its name suggests, this layer, normalizes the output of hidden layers of neural networks. It is widely used in network architecture and generally believed to increase training speed and accuracy. The original authors of the paper suggest that its effectiveness may be due to it fixing a shift in data distribution which occurs in trained layers. Others **?**, have proposed that the effectiveness of batch normalization can be explained otherwise.

RECTIFIED LINEAR UNITS (RELU) ACTIVATION FUNCTION

The ReLU activation function introduces non-linearity and is commonly used in CNNs, which are the basis for GNNs.

$$f(y) = max(0, y) \tag{15}$$

SIGMOID ACTIVATION FUNCTION

The sigmoid function maps input to a value between 0 and 1.

$$f(y) = \frac{1}{1 + e^{-y}} \tag{16}$$

5.2 DEEP Q NETWORK

GLOBAL POOLING

We use mean pooling, similarly to how pooling is used in Convolution Neural Networks, to combine the representation of multiple vertices in a graph into a single representation, using the mean of embeddings of the v vertices. This concept visualized below in Fig. 4.

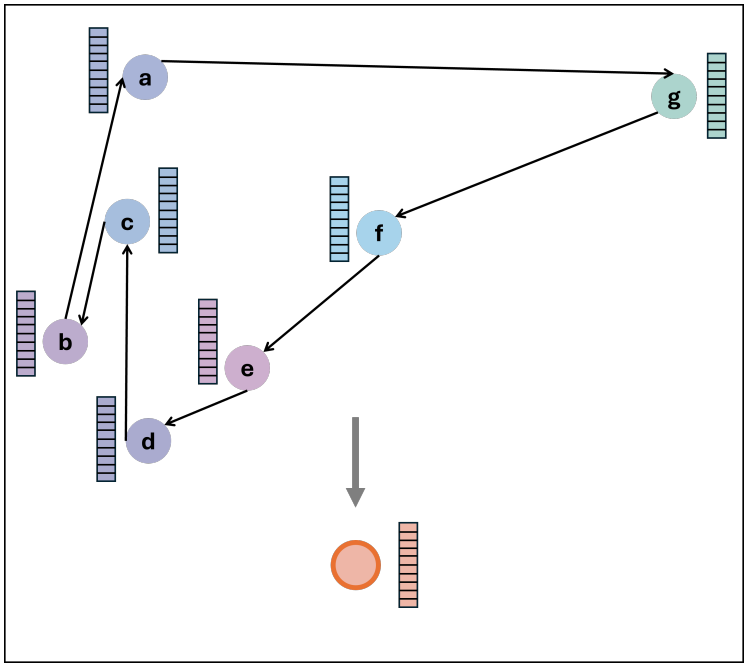


Figure 4: Global Mean Pooling in GNN

FULLY CONNECTED LAYERS

Common in prediction tasks, this layer applies a learnable weight $\vec{w}$ in a linear transformation to the input vector. In the case of our model, the input to the first fully connected layer is m graph embeddings which are each represented by a column vector with dimension d_2. The layer input is then a matrix with size $(m * 1 * d_2)$ and its output has dimension $(m * 1 * d_3)$. Each fully connected layer is followed by a ReLU activation layer to introduce non-linearity. The second fully connected layer reduces the dimension from d_3 to v where v is the number of vertices in the instance and hence the number of possible actions an agent can take. The model returns an estimate of $Q(s, a)$ which has shape $(m * 1 * v)$.

When choosing an action we use the softmax activation function on this output to determine with what probability the agent should select each action. This is the policy used to select the "best" action under exploitation (see Section 4).

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}$$

6 EXPERIMENT RESULTS

6.1 HYPARAMETERS

The hyparameters used in training are shown in the table below. The variables significance to the model are discussed in Section 4.

Table 1: Final Hyparameters

variable	value
d_1	128
d_2	64
d_3	32
dr	0.3
lr	$1e-8$
$l2$	$1e-3$
m	16
b	$1e6$
i_{max}	5000
γ	0.99
ϵ_{min}	0.01
k	1
n	100
δ	$1e-2$
p	5

6.2 TRAINING

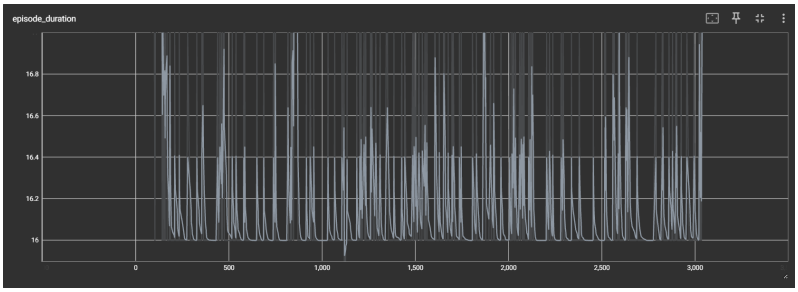


Figure 5: Number of Timesteps (Until All Instances Are Completed) In Each Episode

6.3 TESTING

To evaluate the performance of our reinforcement learning model we hope to test it on instances in the OPLIB, following the experiment design used in existing literature discussed in Section 3. These results are compiled in a table with headings as shown below, where v is the size of the instance, obj is the result of our testing on that instance, $chao$ is the best result produced by Chao et al. (1996b) and gap represents the deviation of our solution from that solution. Chao et al. (1996b) got an average score of 890.6 on the 25 instances with 66 vertices they performed testing on. Their results on the specific instances we used in testing are shown in the table below.

7 DISCUSSION

Training loss plateaus much earlier than expected and the results were not as expected. Perhaps training on bigger batches of data and different hyparameters for the model's hidden dimension were needed.

Table 2: Testing Results

$instance$	v	res	$Chao$
a	66	0	10
l	66	210	915
y	66	390	1655
n	66	235	1070

7.1 Limitations & Future Work

In this project, the instances trained, validated and tested on were all of the same size. This does not indicate whether the model is robust and future work would ideally incorporate working on OP instances of varying sizes in order to ensure that the model will be usable in real life scenarios. Another limitation is that the hyparameters were not tuned systematically, such as with the use of a module like Optuna and so there is no guarantee that the hyparameters used for training were the best to maximize model results. To improve on this work I would complete formal hyparameter tuning using structured experiments. Further computational testing is also needed to try new metrics of comparison between our DRL model and existing heuristics.

References

Egon Balas. The prize collecting traveling salesman problem. *Networks*, 19(6):621–636, 1989.

Irwan Bello, Hieu Pham, Quoc V Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.

Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2): 405–421, 2021.

Sylvain Boussier, Dominique Feillet, and Michel Gendreau. An exact algorithm for team orienteering problems. *4or*, 5:211–230, 2007.

Quentin Cappart, Thierry Moisan, Louis-Martin Rousseau, Isabeau Prémont-Schwarz, and Andre A Cire. Combining reinforcement learning and constraint programming for combinatorial optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 3677–3687, 2021.

I-Ming Chao, Bruce L Golden, and Edward A Wasil. A fast and effective heuristic for the orienteering problem. *European journal of operational research*, 88(3):475–489, 1996a.

I-Ming Chao, Bruce L Golden, and Edward A Wasil. The team orienteering problem. *European journal of operational research*, 88(3):464–474, 1996b.

Dominique Feillet, Michel Gendreau, and Louis-Martin Rousseau. New refinements for the solution of vehicle routing problems with branch and price. *INFOR: Information Systems and Operational Research*, 45(4):239–256, 2007.

Matteo Fischetti, Juan Jose Salazar Gonzalez, and Paolo Toth. Solving the orienteering problem through branch-and-cut. *INFORMS Journal on Computing*, 10(2):133–148, 1998.

Ricardo Gama and Hugo L Fernandes. A reinforcement learning approach to the orienteering problem with time windows. *Computers & Operations Research*, 133:105357, 2021.

Ridvan Gedik, Emre Kirac, Ashlea Bennet Milburn, and Chase Rainwater. A constraint programming approach for the team orienteering problem with time windows. *Computers & Industrial Engineering*, 107:178–195, 2017.

Michel Gendreau, Gilbert Laporte, and Frederic Semet. A branch-and-cut algorithm for the undirected selective traveling salesman problem. *Networks: An International Journal*, 32(4):263–273, 1998.

Bruce Golden, Arjang Assad, and Roy Dahl. Analysis of a large scale vehicle routing problem with an inventory component. *Large scale systems*, 7(2-3):181–190, 1984.

Bruce L Golden, Larry Levy, and Rakesh Vohra. The orienteering problem. *Naval Research Logistics (NRL)*, 34(3):307–318, 1987.

Aldy Gunawan, Hoong Chuin Lau, and Kun Lu. An iterated local search algorithm for solving the orienteering problem with time windows. In *Evolutionary Computation in Combinatorial Optimization: 15th European Conference, EvoCOP 2015, Copenhagen, Denmark, April 8-10, 2015, Proceedings 15*, pp. 61–73. Springer, 2015.

William D Harvey and Matthew L Ginsberg. Limited discrepancy search. In *IJCAI (1)*, pp. 607–615, 1995.

Wouter Kool, Herke Van Hoof, and Max Welling. Attention, learn to solve routing problems! *arXiv preprint arXiv:1803.08475*, 2018.

Gilbert Laporte and Silvano Martello. The selective travelling salesman problem. *Discrete applied mathematics*, 26(2-3):193–207, 1990.

Renata Mansini, M Pelizzari, R WOLFER CALVO, et al. A granular variable neighborhood search heuristic for the tour orienteering problem with time windows. In *Technical Report RT 2006-02-52, Department of Electronics for Automation, University of Brescia*, pp. 1–28. Brescia, 2006.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

Roberto Montemanni, D Weyland, and LM Gambardella. An enhanced ant colony system for the team orienteering problem with time windows. In *2011 international symposium on computer science and society*, pp. 381–384. IEEE, 2011.

Mohammadreza Nazari, Afshin Oroojlooy, Lawrence Snyder, and Martin Takác. Reinforcement learning for solving the vehicle routing problem. *Advances in neural information processing systems*, 31, 2018.

Orienteering Queensland. History of orienteering queensland, n.d. URL `https://oq.orienteering.asn.au/history`.

R Ramesh, Yong-Seok Yoon, and Mark H Karwan. An optimal algorithm for the orienteering tour problem. *ORSA Journal on Computing*, 4(2):155–165, 1992.

Giovanni Righini and Matteo Salani. Decremental state space relaxation strategies and initialization heuristics for solving the orienteering problem with time windows with dynamic programming. *Computers & operations research*, 36(4):1191–1203, 2009.

Hyunchul Tae and Byung-In Kim. A branch-and-price approach for the team orienteering problem with time windows. *International Journal of Industrial Engineering*, 22(2), 2015.

Theodore Tsiligirides. Heuristic methods applied to orienteering. *Journal of the Operational Research Society*, 35:797–809, 1984.

Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.

Pieter Vansteenwegen, Wouter Souffriau, Greet Vanden Berghe, and Dirk Van Oudheusden. Iterated local search for the team orienteering problem with time windows. *Computers & Operations Research*, 36(12):3281–3290, 2009.

Pieter Vansteenwegen, Wouter Souffriau, and Dirk Van Oudheusden. The orienteering problem: A survey. *European Journal of Operational Research*, 209(1):1–10, 2011.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8:279–292, 1992.